universidade de aveiro

deti

Security in Software Engineering

Group 102

Tomás Brás Nº 112665
Afonso Ferreira Nº 113480

Índice

1. Threat Modelling - Context and Objectives
    - System Description
    - Assets
    - Trust Boundaries
    - Data Flow Diagrams
        - DFD Notation Used
        - Level 0 – Context Diagram
        - Level 1 – System Decomposition
        - Level 2 – Detailed View of Appointment Creation
        - Approach
    - LINDDUN Analysis
    - STRIDE Threat Analysis
        - Spoofing
        - Tampering
        - Repudiation
        - Information Disclosure
        - Denial of Service
        - Elevation of Privilege
    - Abuse Cases
    - Attack Trees
2. SAST - Static Application Security Testing
    - Objectives
    - Scope
    - Three-Level SAST Strategy
        - Pre-Commit Level
        - Pull Request Level
    - Nightly Level
    - Why the Three Levels Matter
    - Strengths and Limitations
    - Results
        - Pre-Commit Results
        - Pull Request Results
        - Nightly Results
3. Dynamic Application Security Testing (DAST)
    - Overview
    - Objectives
    - Scope

Two-Level DAST Strategy
Pull Request Level
Nightly Level
Why the Two Levels Matter
Runtime Environment and Tooling
Strengths and Limitations
Results
Pull Request Results

1. Threat Modelling - Context and Objectives

This threat model covers the full-stack clinic management application composed of a React/Vite frontend, a Spring Boot backend, and a PostgreSQL database, orchestrated via Docker Compose. The system manages patient and appointment data for a medical clinic.

Security objectives:

- Protect patient health and contact data (PII/PHI) from unauthorized disclosure
- Prevent unauthorized creation, modification, or deletion of patient and appointment records
- Maintain availability of the system for clinic staff
- Ensure data integrity of medical records

Out of scope: Network-level infrastructure, physical security, end-user workstations.

System Description

Component	Technology	Exposure
Frontend	React 19 + Vite 7, Node dev server	Public (browser)

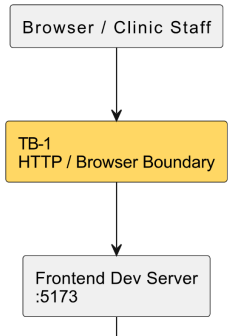
Backend	Java 21 + Spring Boot 4.0.2, port 8080	Host-exposed and proxied via Vite
Database	PostgreSQL 16	Internal only
Orchestration	Docker Compose	Host only

Key observations with security relevance:

- No authentication or authorization layer exists anywhere in the stack
- Backend binds JPA entities directly from HTTP request bodies (no DTOs, no @Valid)
- Frontend runs as a dev server (npm run dev) inside Docker — not a production build
- Auto-seeder populates 120 patients and 520 appointments on empty database startup
- patientId is passed as a query parameter (not in auth context) when creating appointments
- PUT /api/appointments/{id} allows re-assigning patient via body { patient: { id: X } }
- PatientRepository.findByEmail() exists but is unused — email uniqueness is not enforced
- SPECIALTIES and STATUSES are duplicated between frontend and backend with no server-side enum validation
- No server-side pagination, rate limiting, or audit logging is implemented

Assets

The main assets in the system are **patient PII**, which includes name, date of birth, phone number, and email; **appointment records**, which include date and time, specialty, status, and patient linkage; **database credentials** used by the backend to access PostgreSQL; **system availability**, which is necessary for normal clinic operation; and the **audit trail**, which is currently absent and therefore represents an additional risk.



Trust Boundaries

The identified trust boundaries are described as follows:

- **TB-1: HTTP/Browser Boundary**

  No TLS or authentication is used in the current setup.
- **TB-2: Vite Proxy Boundary**

  /api requests are forwarded without authentication or access checks.
- **TB-3: JPA/JDBC Boundary**

  The backend connects to PostgreSQL with a single read/write account and no row-level security.

Data Flow Diagrams

DFD Notation Used

The diagrams use standard DFD-inspired notation adapted to the project context:

- **Rectangle**: external entity
  Represents an actor outside the system boundary that exchanges data with the system.
  In this project, the main external entity is the Patient.

- **Circle / rounded process node**: process
  Represents an internal system function that transforms, validates, stores, or retrieves data.
  Examples in this project are Manage Patient Data, Manage Appointments, and Search and View Information.

- **Cylinder**: data store
  Represents where information is logically stored.
  In this project, the logical data stores are Patient Records and Appointment Records.

- **Arrow / labeled relation**: data flow
  Represents the movement of data between external entities, processes, and data stores.
  The label describes what is flowing, for example Profile data, Appointment request, or Patient record data.

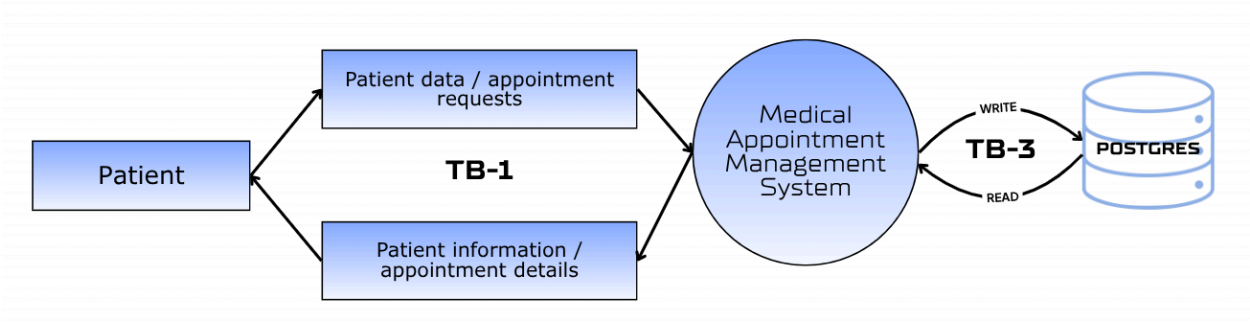
Level 0 – Context Diagram

The Level 0 DFD presents the system as a single process. Its goal is to show the overall system boundary and the high-level data exchanged with the external actor.

In this project:

- the external entity is the Patient
- the entire application is represented as Medical Appointment Management System
- the incoming flows represent patient data and appointment requests
- the outgoing flows represent patient information and appointment details

Additionally, two trust boundaries are identified: TB-1 between the Patient and the system, and TB-3 between the system and the database.



DFD Level 0

Level 1 – System Decomposition

The Level 1 DFD decomposes the single system process into the main internal sub-processes.

In this project, the main sub-processes are:

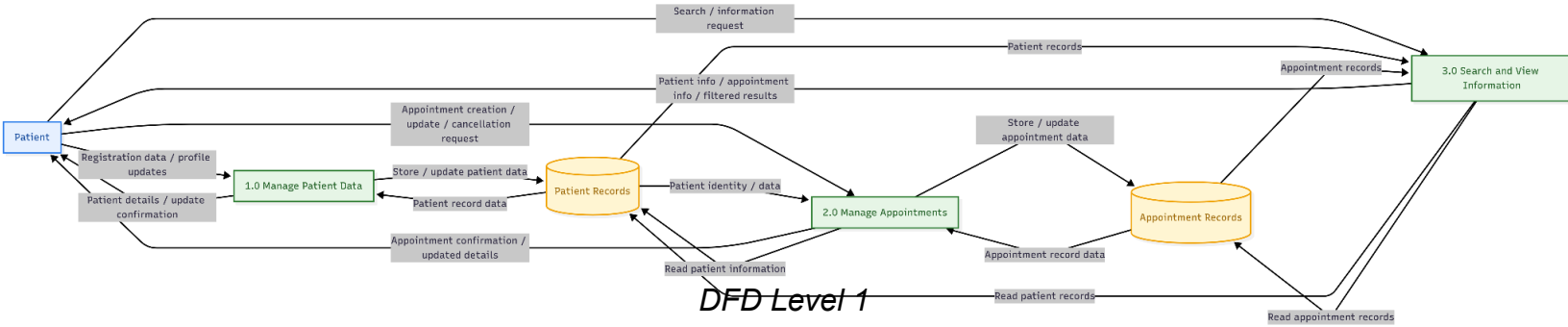
- 1.0 Manage Patient Data
- 2.0 Manage Appointments
- 3.0 Search and View Information

The logical data stores are:

- D1 Patient Records
- D2 Appointment Records

This level shows:

- how patient profile data enters the system
- how appointment requests are processed
- how patient and appointment data are stored and retrieved
- how the search/view process reads records from both data stores



DFD Level 1

Level 2 – Detailed View of Appointment Creation

The Level 2 view focuses on a specific core process instead of the entire system. In this project, the chosen detailed process is Appointment Creation Flow.

Although this level is presented as a DFD decomposition in the slides, it is technically closer to a sequence-style interaction flow. This is acceptable for the project because it explains one of the most important backend operations in detail.

The flow includes:

- Patient
- AppointmentController
- AppointmentService
- PatientRepository
- AppointmentRepository
- Database

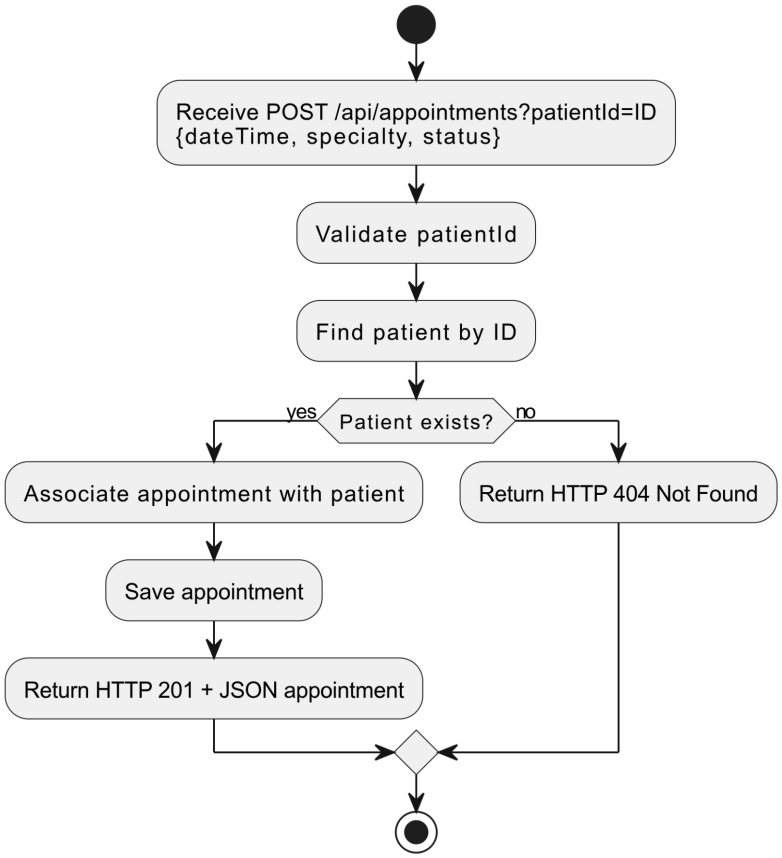
Main steps:

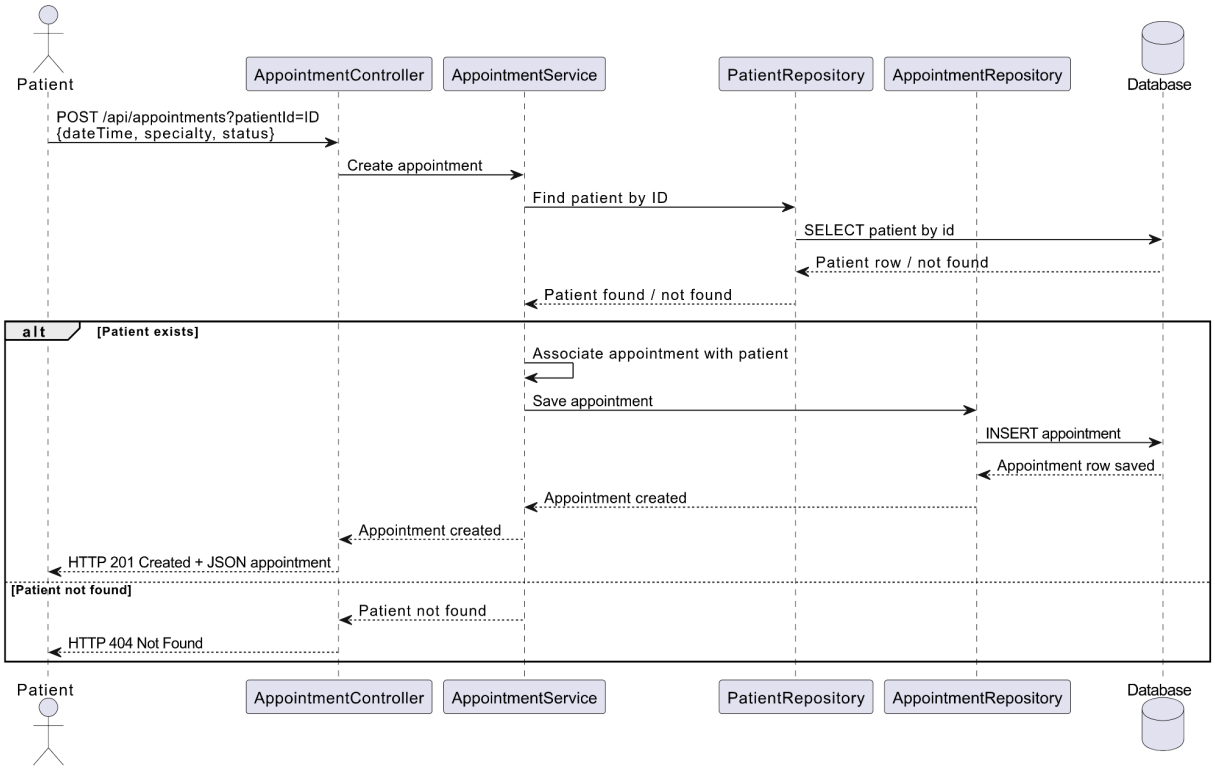
1. The patient sends a request to create an appointment.
2. The controller forwards the request to the service layer.
3. The service validates the patient by checking patientId.
4. If the patient exists, the appointment is associated with that patient and saved.
5. The system returns HTTP 201 Created.
6. If the patient does not exist, the system returns HTTP 404 Not Found.

This level is useful because it shows one of the system's most relevant internal operations with real backend components and data persistence behavior.

At the moment, no exported Level 2 diagram file was found in src/Diagrams. If the Level 2 image is later exported to that folder, it should also be referenced here.

Level 2 - Appointment Creation Flow (Activity)





Approach

This project uses both **LINDDUN** and **STRIDE** as complementary threat modeling methodologies.

LINDDUN was used because the system processes personal and appointment-related data, including patient name, email, phone number, date of birth, appointment date and time, specialty, and status. It is therefore appropriate for analysing privacy risks such as identifiability, linkability, information disclosure, and the inference of sensitive **health**-related patterns.

STRIDE was used to analyse broader security threats affecting the application, namely risks related to spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege.

Using both models provides a more complete analysis, covering both privacy risks and technical security threats relevant to the project.

LINDDUN Analysis

LINDDUN Category	Relevance	Description	Detail
Identifiability	High	Patients can be directly identified from stored records, which include name, email, phone number, and date of birth.	All fields are returned in full via `GET /api/patients`. Sequential integer IDs make enumeration trivial.
Linkability	High	Patient identity can be linked to appointment records, enabling inference of health patterns such as specialty history and visit frequency.	`Appointment` is linked to `Patient`. The search API filters by `patientName`, `date`, `status`, and `specialty`, all without authentication.
Disclosure of Information	High	Unauthenticated endpoints expose full patient and appointment data to any	`GET /api/patients` and `GET`

		caller, with no field filtering or access scoping.	`/api/appointments` require no credentials and return complete entity data.
Detectability	Medium	An attacker can confirm whether a named individual is a patient and probe which records exist by observing API responses.	Sequential IDs and 200/404 differences on `/{id}` endpoints expose record existence. `GET /api/appointments?patientName=X` confirms patient presence with no authentication.
Non-repudiation	Medium	No authentication or audit logging exists, so data access and modifications cannot be attributed to any actor.	The backend has no session tracking or audit events. Any CRUD operation is permanently unattributable.
Unawareness	Medium	Patients have no way to know how their data is processed or to exercise their data subject rights.	No consent flow, privacy notice, or rights-handling workflow exists. GDPR Articles 15–17 cannot be fulfilled.
Non-compliance	High	The system has structural GDPR violations applicable to health data, exposing the clinic to regulatory liability.	No lawful basis for processing; explicit consent for health data;no retention policy;no breach notification capability;

STRIDE Threat Analysis

Spoofing

ID	Threat	Component	Severity	Detail

S-01	Any user can impersonate any clinic staff member **no authentication exists**	Backend API	Critical	No Spring Security configuration or auth middleware is present anywhere in the stack.
S-02	Attacker can forge patientId query parameter to create appointments under any patient	POST /api/appointments	High	AppointmentController.create() accepts patientId from the request and AppointmentService.create() trusts it after findById().

Tampering

ID	Threat	Component	Severity	Detail
T-01	Any unauthenticated user can PUT to /api/appointments/{id} and change patient linkage, date, status, or specialty	Appointment API	Critical	AppointmentService.update() updates all mutable fields and optionally swaps the linked patient.
T-02	Any unauthenticated user can PUT to /api/patients/{id} and modify patient PII	Patient API	Critical	PatientService.update() directly replaces name, dateOfBirth, phoneNumber, and email.
T-03	Direct entity binding accepts nested relationship data from clients, increasing the risk	Backend (no DTOs)	Medium	Controllers deserialize directly into JPA entities instead of

	of unintended writable fields and relationship manipulation			constrained request DTOs.
T-04	SPECIALTIES and STATUSES not validated server-side — attacker can persist arbitrary strings	Appointment entity	Medium	specialty and status are plain String fields with no enum, regex, or bean validation.
T-05	Email field has no uniqueness constraint enforced by the application	Patient API	Medium	findByEmail() exists in the repository but is not used during create or update flows.

Repudiation

ID	Threat	Component	Severity	Detail
R-01	No audit logging — any CRUD operation (including deletion of patient records) leaves no trace	Entire backend	High	No audit service, interceptor, event sink, or persistence of access/change history is implemented.
R-02	No authentication means no actor identity	Entire system	Critical	Requests are anonymous end-to-end, so even

	is ever recorded in logs			server logs cannot attribute actions to a real actor.
R-03	Cascade delete on Patient removes all linked appointments with no log or recovery path	Patient delete API	High	Patient uses cascade = CascadeType.ALL and orphanRemoval = true on appointments.

Information Disclosure

ID	Threat	Component	Severity	Detail
I-01	Full patient PII returned in all GET /api/patients responses — no field-level filtering or RBAC	Patient API	High	The API returns entity objects directly, including patient PII and linked appointment collections.
I-02	No pagination — GET /api/patients returns all 120+ records in one response	Patient API	Medium	PatientService.getAll() calls findAll() with no limit, page, or field projection.
I-03	Unhandled error responses may disclose implementation details, depending on runtime configuration	Backend	Low	No explicit production error-handling policy is configured in the repository.

I-04	Database credentials stored in .env file — leaked if repo is made public	Config	High	docker-compose.yml reads POSTGRES_USER and POSTGRES_PASSWORD from environment variables backed by .env.
I-05	Dev server (npm run dev) exposes development assets and client-side implementation details	Frontend container	Medium	The frontend container runs vite directly instead of serving a production build through a hardened web server.
I-06	Backend port 8080 may be directly accessible bypassing the Vite proxy	Docker networking	Medium	docker-compose.yml publishes the backend port to the host, not only to the internal Docker network.

Denial of Service

ID	Threat	Component	Severity	Detail
D-01	No rate limiting — attacker can flood POST /api/appoi	Backend API	High	There is no throttling, quota, or back-pressure mechanism at frontend,

	ntments to exhaust DB connections			backend, or gateway level.
D-02	No pagination on GET /api/appointments with JPA Specification — large result sets can exhaust memory	Appointment search	Medium	appointmentRepository.findAll(spec) returns the full matching set in memory.
D-03	DELETE /api/patients/{id} cascades to all appointments — single request can delete hundreds of records	Patient delete	High	A single patient deletion removes the parent record and all dependent appointments in one operation.

Elevation of Privilege

ID	Threat	Component	Severity	Detail
E-01	No roles or permissions — all API operations are equally accessible to any caller	Entire backend	Critical	There is no RBAC, ownership check, or record-level authorisation anywhere in the

				backend flow.
E-02	Client-controlled nested objects can influence backend relationships during updates, bypassing intended UI constraints	Backend (no DTOs)	High	The update flow trusts nested patient.id in request bodies to change ownership of appointments.
E-03	PUT /api/appointments/{id} re-assigns patient — allows transferring medical records between patients	Appointment service	High	AppointmentService.update() resolves the provided patient ID and rebinds the appointment if found.

Abuse Cases

Privacy and security threats were identified using the LINDDUN framework, suited for systems processing personal and health-related data.

LINDDUN categories:

Linking · Identifying · Non-repudiation · Detecting · Disclosure · Unawareness · Non-compliance.

ID: AS-01

Title: Unauthorized Access to Patient Data

LINDDUN Principal and Risk: Disclosure - **CRITICAL**

- **Abuse Case**:
  - **As a** malicious external user,
  - **I want to** access another patient's personal data
  - **so that I** can obtain sensitive information such as name, email, phone number, date of birth, and linked appointment data.
- **How the abuse works**:
  The attacker sends direct requests to /api/patients and /api/appointments and retrieves records without any access control, as no authentication mechanism exists in the system.
- **Impact**:

Exposure of personal and appointment-related data, leading to a privacy breach.

ID: AS-02

Title: Link Patient Identity to Appointment History

LINDDUN Principal and Risk: Linking - **CRITICAL**

- **Abuse Case**:
  - **As a** malicious external user,
  - **I want to** associate patient identity data with appointment records
  - **so that I** can infer sensitive patterns such as recurring specialties and possible health-related conditions.
- **How the abuse works**:
  The attacker correlates the full patient list with appointment records through linked identifiers. Both endpoints are unauthenticated and return all records without pagination, making full profiling possible in two requests.
- **Impact**:
  Sensitive health patterns can be inferred from metadata alone, increasing patient profiling risk beyond what any single record reveals.

ID: AS-03

Title: Directly Identify a Patient from Stored Records

LINDDUN Principal and Risk: Identifying - **CRITICAL**

- **Abuse Case**:
  - **As a** malicious external user,
  - **I want to** use personal identifiers stored in the system
  - **so that I** can directly identify a patient and associate their appointment information with a real individual.
- **How the abuse works**:
  The attacker accesses records containing direct identifiers (name, email, phone, date of birth). Patient IDs are sequential integers starting at 1, making full enumeration via /api/patients/{id} trivial with no rate limiting or authentication to prevent it.
- **Impact**:
  A real individual can be directly identified and their full appointment history retrieved, increasing the severity of exposure.

ID: AS-04

Title: Infer a Patient's Health Condition from Appointment Metadata

LINDDUN Principal and Risk: Detecting - **CRITICAL**

- **Abuse Case**:
  - **As a** malicious external user,
  - **I want to** observe the specialty and status fields of a patient's appointments
  - **so that I** can infer sensitive health information without ever accessing a medical record directly.
- **How the abuse works**:
  The attacker filters GET /api/appointments?patientName=X to retrieve the full appointment history of a named individual. Combinations such as repeated "Psychiatry" visits with status "NO_SHOW" reveal sensitive patterns without any explicit diagnosis data being present.
- **Impact**:
  Sensitive health conditions may be inferred from observable metadata, constituting a privacy violation even in the absence of explicit medical data.

I**D**: AS-05

Title: Modify or Reassign Patient Appointment Records

LINDDUN Principal and Risk: Disclosure - **HIGH**

- **Abuse Case**:
  - **As a** malicious external user,
  - **I want to** change the date, status, specialty, or linked patient of an appointment
  - **so that I** can corrupt legitimate medical records.
- **How the abuse works**:
  The attacker sends an unauthenticated PUT request to /api/appointments/{id}. Including "patient": {"id": X} in the body causes the service to silently reassign the appointment to a different patient, with no authorization check, no validation, and no audit log entry.
- **Impact**:
  Medical records may be altered or linked to the wrong patient, directly affecting clinical integrity and patient safety.

ID: AS-06

Title: Destroy Patient Records Through Cascade Deletion

LINDDUN Principal and Risk: Non-Compliance - **CRITICAL**

- **Abuse Case**:
  - **As a** malicious external user,
  - **I want to** remove patient records
  - **so that I** can cause irreversible data loss and disrupt normal clinical operation.
- **How the abuse works**:
  The attacker sends unauthenticated DELETE requests to /api/patients/{id}. Due to cascade deletion (orphanRemoval = true), every appointment linked to that patient is also permanently destroyed. IDs are sequential, so iterating from 1 to N wipes the entire database. No confirmation, audit log, or recovery path exists.
- **Impact**:
  All patient PII and linked appointment history is permanently lost with no recovery mechanism. The clinic cannot detect, investigate, or report the incident.

ID: AS-07

Title: Perform Undetectable Actions Due to Absence of Audit Logging

LINDDUN Principal and Risk: Non-repudiation - **CRITICAL**

- **Abuse Case**:
  - **As a** malicious internal user or external attacker,
  - **I want to** create, modify, or delete patient and appointment records
  - **so that** my actions leave no trace and cannot be attributed to me after the fact.
- **How the abuse works**:
  The system has no authentication (so no actor identity is ever recorded), no audit log, and no access log at any layer. Any CRUD operation on any record is permanently undetectable once performed.
- **Impact**:
  Malicious or accidental changes cannot be investigated or attributed. The clinic cannot comply with GDPR.

ID: AS-08

Title: Overload the API with Repeated Requests

LINDDUN Principal and Risk: Non-compliance - **HIGH**

- **Abuse Case**:
  - **As an** attacker,
  - **I want to** flood patient and appointment endpoints with repeated requests
  - **so that I** can make the system slow or unavailable for legitimate users.
- **How the abuse works**:
  The attacker automates repeated API requests. No rate limiting exists, and GET /api/appointments with no filters triggers a full table scan with no pagination, exhausting the database connection pool (default HikariCP max ~10 connections) under moderate load.
- **Impact**:
  The system becomes slow or unavailable for normal clinical use, impacting patient care.

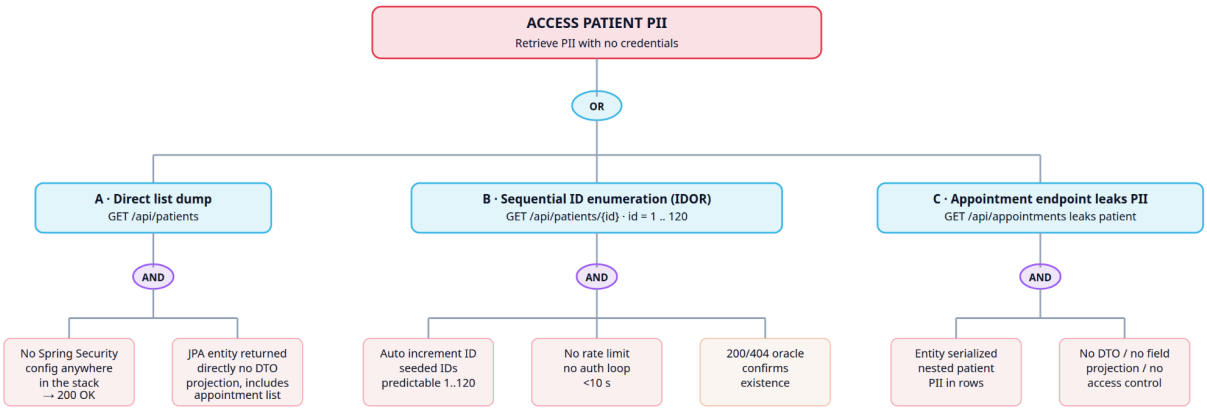
Attack Trees

Attack trees were used to decompose each high-level attacker goal into concrete, testable attack paths and to make the underlying assumptions, preconditions, and impacts explicit.

Each tree starts with a single abuse story goal and expands into multiple branches that represent distinct ways an attacker could achieve it through the exposed API and frontend. The trees help map specific endpoints, missing controls, and data flows to real-world outcomes, and they provide a structured basis for prioritizing tests by confidentiality, integrity, and availability impact.

In total, eight attack trees were produced, one for each abuse story, and they were used to drive the selection and ordering of the pentest scenarios.

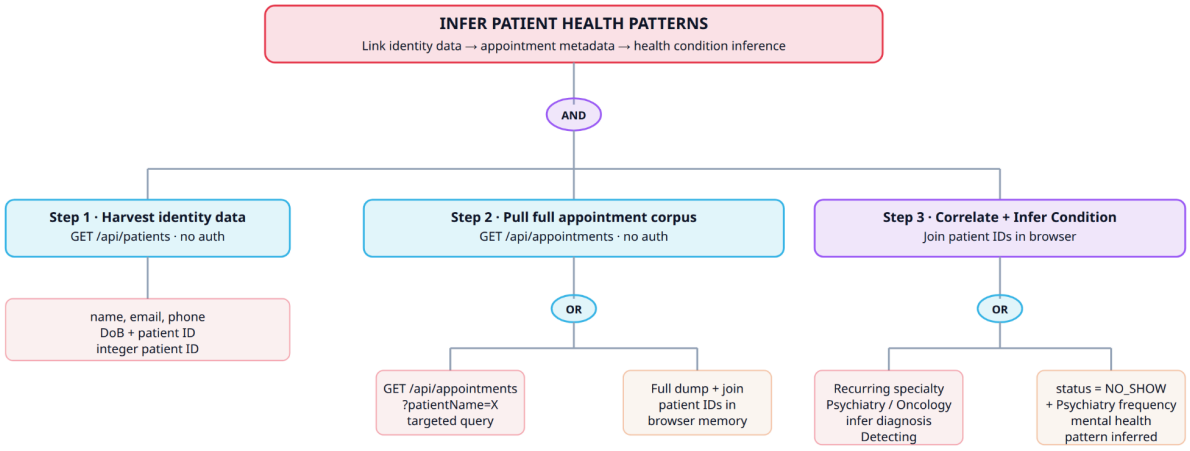
1. Unauthorized Access to Patient Data



Title	Unauthorized Access to Patient Data

Description	An unauthenticated caller retrieves full patient PII and linked appointments. Three independent attack paths all succeed without any credential.
STRIDE	Info Disclosure, Spoofing
LINDDUN	Disclosure
Abuse Stories	AS-01, I-01, S-01, E-01
Severity	**CRITICAL**

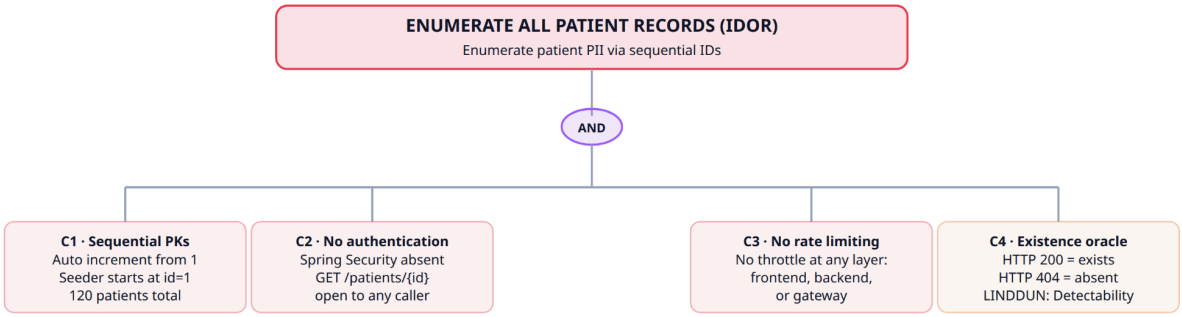
2. Link Patient Identity to Health Patterns



Title	Link Patient Identity to Health Patterns (AS-02 / AS-04)
Description	Attacker correlates patient identity with appointment metadata (specialty, status, frequency) to infer health conditions without accessing any medical record directly.
STRIDE	Information Disclosure
LINDDUN	Linking, Detecting

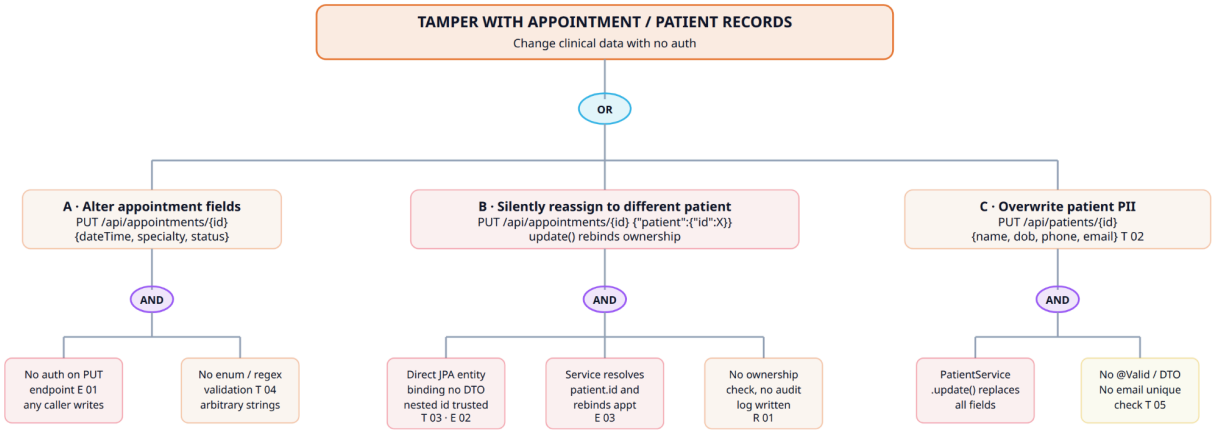
Abuse Stories	AS-02, AS-04, I-01, I-02
Severity	**CRITICAL**

3. Direct Patient Identification via IDOR



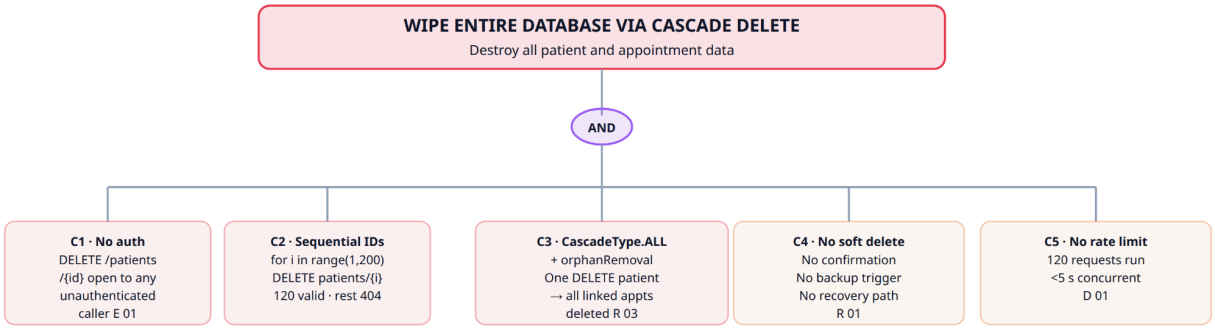
Title	Direct Patient Identification via IDOR (AS-03)
Description	Sequential integer IDs with no rate limiting allow trivial enumeration of all 120 patient records. The 200/404 status difference acts as an existence oracle.
STRIDE	Info Disclosure, IDOR
LINDDUN	Identifying, Detectability
Abuse Stories	AS-03, I-01, D-01
Severity	**CRITICAL**

4. Modify or Reassign Appointment Records



Title	Modify or Reassign Appointment Records (AS-05)
Description	Unauthenticated PUT requests allow any caller to alter appointment fields or silently rebind an appointment to a different patient, corrupting medical records with no audit trail.
STRIDE	Tampering, EoP
LINDDUN	Disclosure
Abuse Stories	AS-05, T-01-T-04, E-02, E-03
Severity	**HIGH**

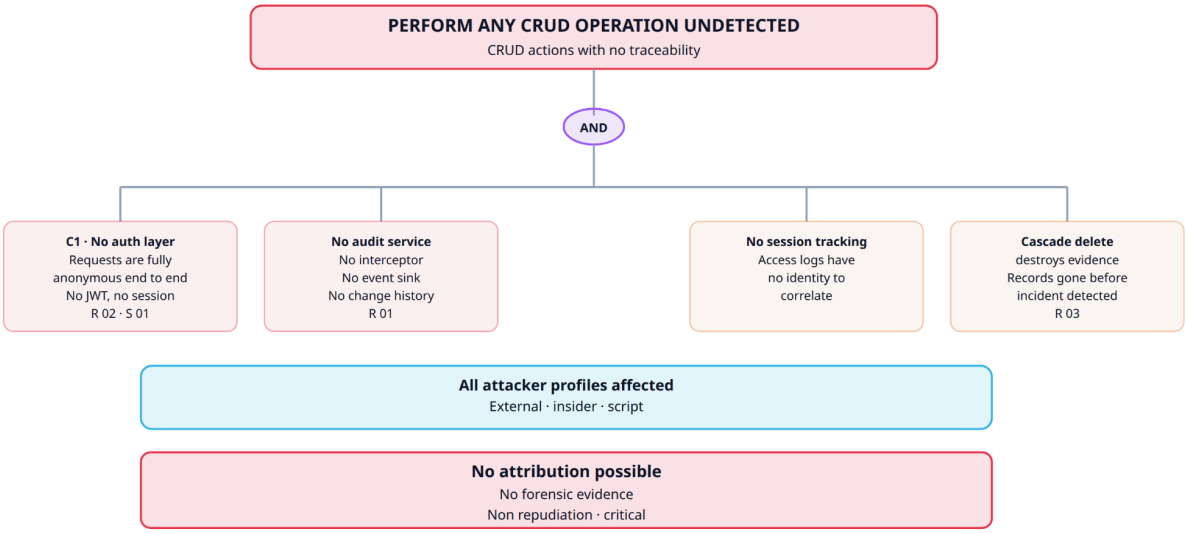
5. Destroy Patient Records via Cascade Deletion



Title	Destroy Patient Records via Cascade Deletion (AS-06)

Description	DELETE /api/patients/{id} triggers CascadeType.ALL + orphanRemoval, permanently deleting the patient and all linked appointments. Sequential IDs enable full database wipe with ~120 requests.
STRIDE	Tampering, DoS, Repudiation
LINDDUN	Non-Compliance
Abuse Stories	AS-06, D-03, R-01, R-02, R-03, E-01
Severity	**CRITICAL**

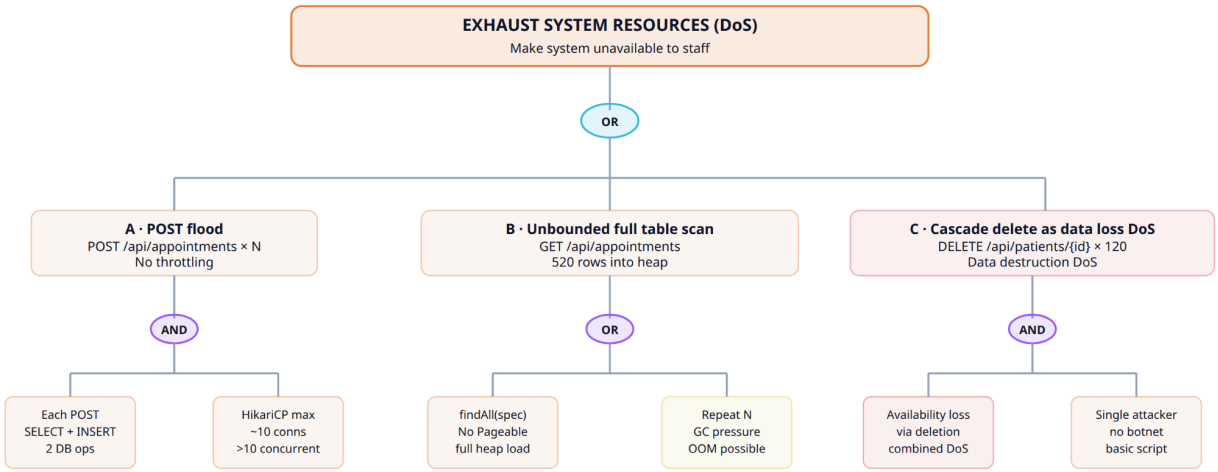
6. Perform Undetectable Actions



Title	Perform Undetectable Actions (AS-07)
Description	The total absence of authentication, session tracking, and audit logging means any CRUD operation — by any actor — leaves zero forensic evidence and cannot be attributed or investigated.
STRIDE	Repudiation

LINDDUN	Non-repudiation
Abuse Stories	AS-07, R-01, R-02, R-03
Severity	**CRITICAL**

7. Overload API / Denial of Service



Title	Overload API / Denial of Service (AS-08)
Description	No rate limiting at any layer combined with unbounded queries allows exhaustion of the HikariCP connection pool (~10 connections default), rendering the system unavailable.
STRIDE	Denial of Service
LINDDUN	Non-Compliance
Abuse Stories	AS-08, D-01, D-02, D-03
Severity	**HIGH**

SAST - Static Application Security Testing

Static Application Security Testing (SAST) was used in this project to analyze source code, security configuration, and build artifacts without relying on runtime exploitation. Its purpose is to identify insecure coding patterns, exposed secrets, vulnerable dependencies, and insecure project configurations as early as possible in the development lifecycle.

For this project, SAST is not treated as a single scan executed at one point in time. Instead, it is organized as a three-level process:

- **pre-commit**, for fast local feedback before code is committed
- **pull request**, for gated review before changes are merged
- **nightly**, for broader recurring analysis with deeper coverage

This structure creates a layered security process in which lightweight checks happen early and frequently, while deeper scans run later with more time and broader scope.

Objectives

The SAST process in this project was designed to:

- detect insecure code patterns in the Java Spring Boot backend and React/Vite frontend
- identify secrets, dependency risks, and misconfigurations before deployment
- prevent obvious security issues from reaching the main branch
- provide evidence through GitHub Actions runs, uploaded artifacts, and summary reports
- support the wider application security assessment together with DAST and manual pentesting

Scope

The static analysis scope covers the Clinic Project backend source code in **`src/backend`**, the frontend source code in **`src/frontend`**, the repository-level security configuration, the GitHub Actions workflow logic, and the Docker-related build artifacts and container contents. Together, these elements define the code, configuration, and packaged components that were included in the SAST process.

Three-Level SAST Strategy

Pre-Commit Level

The first level is local and developer-facing. It is defined through .pre-commit-config.yaml and is intended to stop obvious issues before they even reach GitHub.

At this level, the repository uses:

- **Gitleaks** to detect secrets and sensitive tokens in staged changes
- **Semgrep** in a fast security-focused mode using p/security-audit and the custom project rules in **.github/config/security/semgrep.yml**

This level should be understood as the fastest security checkpoint. It is intentionally lightweight and optimized for quick developer feedback. The purpose is not exhaustive coverage, but early prevention of high-signal issues such as leaked credentials or obvious insecure code patterns.

Pull Request Level

The second level runs on pull requests through sast.yml. This is the main review and enforcement layer before code is merged.

At PR level, the repository performs:

- **Gitleaks** over the Git range introduced by the pull request
- **Semgrep** with an enforcing scan for **ERROR** findings and an advisory scan for WARNING and INFO
- **ESLint Security** for frontend-oriented security linting
- **Trivy** filesystem and backend dependency scans
- **SonarQube** analysis with a project-specific quality gate

The **quality gate** for **SonarQube** has the following rules:

- no new bugs
- no new vulnerabilities
- all new security hotspots must be reviewed
- no significant maintainability degradation in new code
- duplication in new code must stay below 3%

Then a **SAST Security Summary** is posted back to the pull request

This layer is broader and stricter than pre-commit. It is designed to block unsafe code from being merged, upload security artifacts for later review, and present reviewers with a consolidated security view inside GitHub.

Nightly Level

The third level runs through sast-nightly.yml. It is scheduled and intended to provide deeper, recurring coverage over the repository outside the time pressure of a single developer commit or PR.

At nightly level, the repository performs:

- **CodeQL** for **java-kotlin and javascript-typescript**
- **Semgrep** with broader rule sets including p/owasp-top-ten, p/java, p/javascript, and custom rules
- **Trivy** image scanning over the backend and frontend Docker images

This layer is important because some checks are too expensive or too broad to run as often as pre-commit hooks. Nightly scanning acts as a continuous baseline and can surface issues that were missed or not fully evaluated during faster earlier stages.

Why the Three Levels Matter

The three levels serve different purposes:

- **pre-commit** minimizes developer mistakes before code is pushed
- **pull request** provides security gating and reviewer visibility before merge

- **nightly** provides deeper scheduled assurance and broader long-term monitoring

This separation is useful because not all security checks have the same tradeoff between speed, coverage, and noise. Fast local checks reduce friction, PR checks enforce minimum security quality, and nightly scans provide depth without slowing down day-to-day development.

Strengths and Limitations

One of the main strengths of this model is that security checks are distributed across different moments of the development process instead of being concentrated in a single scan. Pull request checks create clear accountability at merge time, while nightly scans provide broader and deeper recurring coverage without slowing down day-to-day development. In addition, GitHub Actions artifacts, workflow logs, and generated summaries make the results easier to review, document, and present as evidence.

On the other hand, static analysis can produce false positives, and some findings may not be exploitable in practice. The effectiveness of the pre-commit layer also depends on developers having the tooling correctly installed and enabled locally. Furthermore, SAST alone cannot fully assess runtime behavior, business logic flaws, or the real exploitability of certain issues, and image or dependency findings often require manual interpretation to understand their actual impact on the project. For that reason, SAST should be seen as one component of a broader security assessment rather than a complete security validation on its own.

Results

This section should be completed by presenting evidence from each of the three SAST levels.

Pre-Commit Results

Pre-Commit Hook Execution

```
(.venv) alof@alof-Victus-by-HP-Gaming-Laptop-15-fa0xxx:~/Desktop/SES/project/softwate-baseline-ses25_102$ pre-commit run --all-files
Detect hardcoded secrets.................................................Passed
semgrep (security - fast)................................................Passed
```

Pre-Commit Configuration

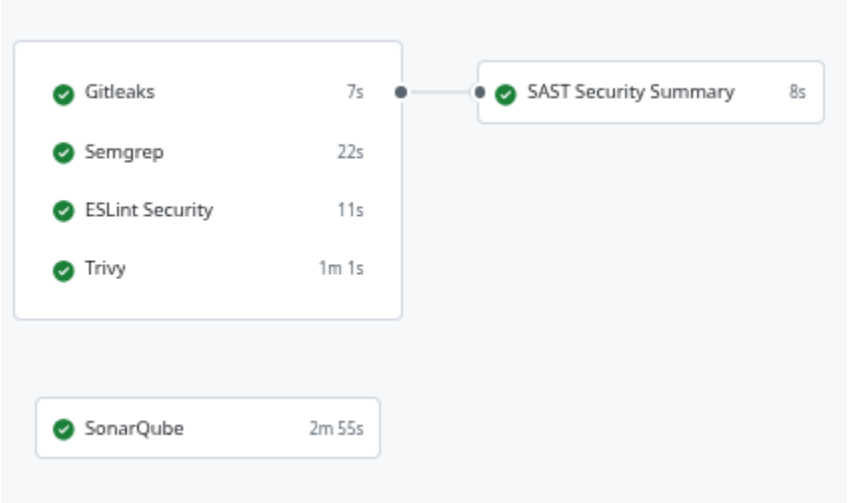
```yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.30.1
    hooks:
      - id: gitleaks
        args:
          - --config=.github/config/security/gitleaks.toml

  - repo: https://github.com/semgrep/pre-commit
    rev: v1.145.0
    hooks:
      - id: semgrep
        name: semgrep (security - fast)
        args:
          - --config
          - p/security-audit
          - --config
          - .github/config/security/semgrep.yml
          - --severity
          - ERROR
          - --error
          - --disable-version-check
          - --metrics=off
          - --use-git-ignore
```

Pull Request Results

During pull request validation, the repository executed the security controls defined in the PR SAST workflow to assess the changes before merge. According to the pull request results, **Gitleaks** passed without reporting blocking secret exposure, **Semgrep** passed its enforcement gate but produced one non-blocking medium advisory finding, **ESLint Security** passed while reporting two advisory warnings, and **Trivy** passed while reporting two low-severity advisory findings related to missing **HEALTHCHECK** instructions in the Dockerfiles. **SonarQube** also passed, with both the scan and the quality gate reported as successful and no bugs, vulnerabilities, or security hotspots detected in the new code for that pull request. In addition, a **SAST Security Summary** comment and a **SonarQube Analysis Report** comment were posted automatically back to the pull request, which provided reviewers with a consolidated security view directly within GitHub.

PR Workflow Overview



PR Security Summary Comment

SAST Security Summary

Tool	Gate Status	Advisory Findings
Gitleaks	Passed	-
Semgrep	Passed	1 medium
ESLint Security	Passed	2 warning
Trivy	Passed	2 low

Non-Blocking Issues Found

These findings do not fail the merge gate, but they should still be reviewed.

Semgrep Advisory Findings

Non-blocking findings: **1** (1 medium)

- **Medium** `[python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected]` Detected a dynamic value being used with urllib. urllib supports 'file://' schemes, so a dynamic value controlled by a malicious actor may allow them to read arbitrary files. Audit uses of urllib calls to ensure user data cannot control the URLs, or consider using the 'requests' library instead. in `.github/scripts/build_sonarqube_pr_comment.py:24`

ESLint Advisory Findings

Findings: **2** (2 warning)

- **Warning** `[security/detect-object-injection]` Generic Object Injection Sink in `/home/runner/work/softwate-baseline-ses25_102-mirror/softwate-baseline-ses25_102-mirror/src/frontend/src/components/AppointmentsPanel.jsx:145`
- **Warning** `[security/detect-object-injection]` Generic Object Injection Sink in `/home/runner/work/softwate-baseline-ses25_102-mirror/softwate-baseline-ses25_102-mirror/src/frontend/src/constants/clinicOptions.js:32`

Trivy Advisory Findings

Non-blocking findings: **2** (2 low)

- **Low** `DS-0026` : Artifact: src/backend/Dockerfile Type: dockerfile Vulnerability DS-0026 Severity: LOW Message: Add HEALTHCHECK instruction in your Dockerfile Link: DS-0026
- **Low** `DS-0026` : Artifact: src/frontend/Dockerfile Type: dockerfile Vulnerability DS-0026 Severity: LOW Message: Add HEALTHCHECK instruction in your Dockerfile Link: DS-0026

Workflow Links

- Open workflow run and artifacts

SonarQube Analysis Report

Quality Gate: Passed

Check	Status
Scan execution	success
Quality gate check	success

Code Metrics

New Code (This PR)

Metric	Value
Bugs	**0**
Vulnerabilities	**0**
Code Smells	**0**
Security Hotspots	**0**
Coverage	**N/A**
Duplications	**0.0%**
Lines of Code	**0**

Overall Project

Metric	Value
Bugs	0
Vulnerabilities	0
Code Smells	89
Security Hotspots	0
Coverage	0.0%
Duplications	10.0%
Lines of Code	2938

Quality Gate Requirements

Your PR will be checked for:

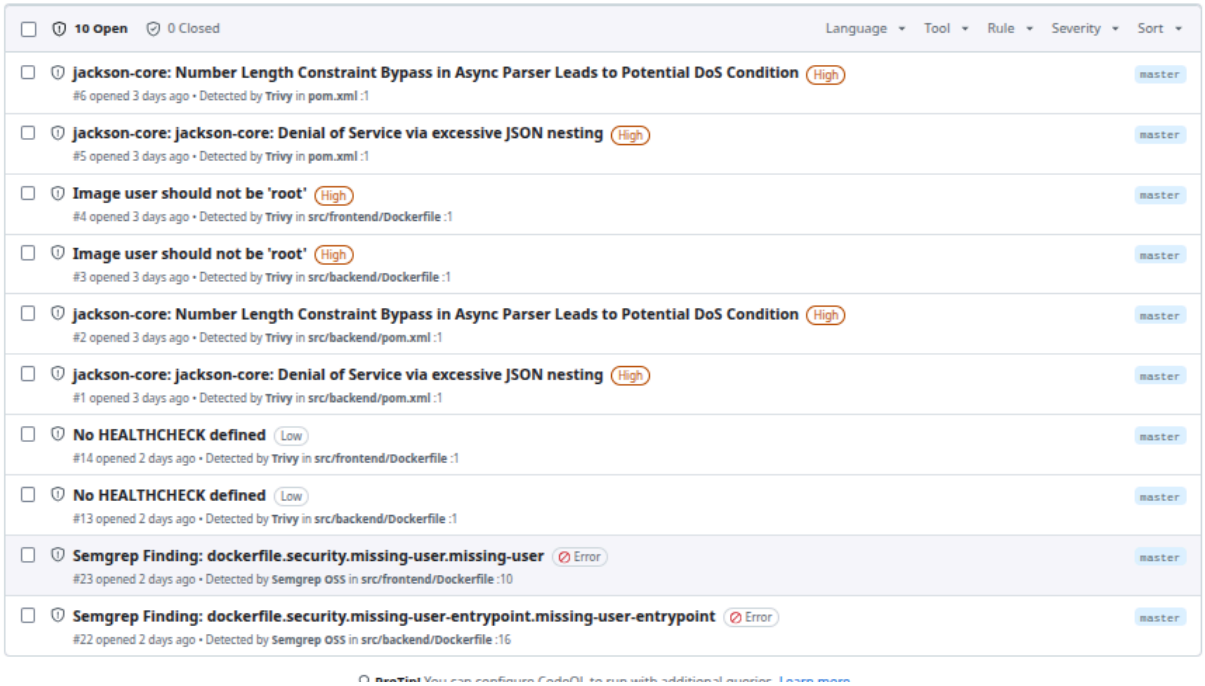
- No bugs in new code
- No vulnerabilities in new code
- All security hotspots reviewed
- No code smells in new code
- Less than 3% code duplication

SonarCloud Analysis Comment

PR Artifacts and Security tab

eslint-security-report	2.16 KB
gitleaks-pr-report	6.9 KB
semgrep-pr-advisory-report	35.2 KB
semgrep-pr-report	11.3 KB
trivy-backend-advisory-report	521 Bytes
trivy-backend-report	519 Bytes
trivy-fs-advisory-report	1.05 KB
trivy-fs-report	464 Bytes

Some of these artifacts are `.sarif` files, which allowed us to upload the scan results to GitHub's Security tab.

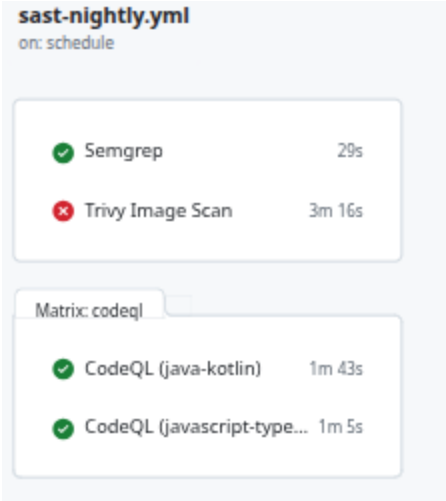


Nightly Results

From the visible results, both CodeQL and Semgrep completed and produced analysis output, while the failure was caused by the Trivy image scanning stage. The run generated a Semgrep report as well as separate Trivy reports for the backend and frontend images. This means the nightly workflow was able to complete the code-focused analysis successfully, but still identified issues in the container image layer that were significant enough to fail the scan.

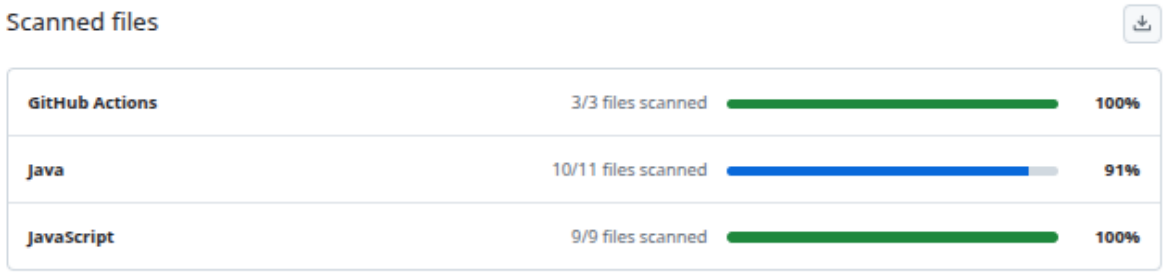
Overall, this result shows the value of the nightly level because even tthough the pre-commit and PR stages completed successfully a more deeply scan can still detect additional security problems and provide a deeper view of the project's overall security posture.

Nightly Workflow Overview



CodeQL Results

(No problems found)



Semgrep Nightly Summary

Semgrep Nightly Summary

Status
Passed

Semgrep Findings

Status: Passed

Severity	Count
Error	0
Warning	0
Note	0
Total	0

No Semgrep findings were reported.

Trivy Image Scan Results

Trivy Image Scan Summary

Image	Status
Backend	**failed**
Frontend	**failed**

Backend Image Vulnerabilities

Package	Vulnerability	Severity	Installed	Fixed
org.apache.tomcat.embed:tomcat-embed-core	CVE-2026-24734	HIGH	11.0.15	11.0.18, 10.1.52, 9.0.115

Frontend Image Vulnerabilities

Package	Vulnerability	Severity	Installed	Fixed
libcrypto3	CVE-2025-15467	CRITICAL	3.5.4-r0	3.5.5-r0
libcrypto3	CVE-2025-69419	HIGH	3.5.4-r0	3.5.5-r0
libcrypto3	CVE-2025-69421	HIGH	3.5.4-r0	3.5.5-r0
libssl3	CVE-2025-15467	CRITICAL	3.5.4-r0	3.5.5-r0
libssl3	CVE-2025-69419	HIGH	3.5.4-r0	3.5.5-r0
libssl3	CVE-2025-69421	HIGH	3.5.4-r0	3.5.5-r0
zlib	CVE-2026-22184	HIGH	1.3.1-r2	1.3.2-r0
cross-spawn	CVE-2024-21538	HIGH	7.0.3	7.0.5, 6.0.6
flatted	CVE-2026-32141	HIGH	3.3.3	3.4.0
flatted	CVE-2026-33228	HIGH	3.3.3	3.4.2
glob	CVE-2025-64756	HIGH	10.4.2	11.1.0, 10.5.0
minimatch	CVE-2026-26996	HIGH	3.1.2	10.2.1, 9.0.6, 8.0.5, 7.4.7, 6.2.1, 5.1.7, 4.2.4, 3.1.3
minimatch	CVE-2026-27903	HIGH	3.1.2	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.3
minimatch	CVE-2026-27904	HIGH	3.1.2	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.4
minimatch	CVE-2026-26996	HIGH	9.0.5	10.2.1, 9.0.6, 8.0.5, 7.4.7, 6.2.1, 5.1.7, 4.2.4, 3.1.3
minimatch	CVE-2026-27903	HIGH	9.0.5	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.3
minimatch	CVE-2026-27904	HIGH	9.0.5	10.2.3, 9.0.7, 8.0.6, 7.4.8, 6.2.2, 5.1.8, 4.2.5, 3.1.4
picomatch	CVE-2026-33671	HIGH	4.0.3	4.0.4, 3.0.2, 2.3.2
rollup	CVE-2026-27606	HIGH	4.57.1	2.80.0, 3.30.0, 4.59.0
tar	CVE-2026-23745	HIGH	6.2.1	7.5.3

Dynamic Application Security Testing (DAST)

Overview

Dynamic Application Security Testing (DAST) was used in this project to assess the running application from the outside, through its exposed web interface and HTTP endpoints, without relying on access to the source code during execution. Its purpose is to identify runtime security issues such as missing security headers, weak transport settings, unsafe default behavior, exposed administrative or debugging functionality, and other vulnerabilities that become visible only when the application is deployed and reachable over the network.

For this project, DAST is not treated as a one-time scan. Instead, it is organized as a two-level process. The first level runs on pull requests and provides fast validation before changes are merged. The second level runs nightly and provides broader recurring coverage against a stable target branch. This structure allows the project to combine merge-time feedback with continuous runtime monitoring.

Objectives

The DAST process in this project was designed to detect runtime security weaknesses in the deployed application, validate the behavior of the frontend and backend when exposed through HTTP, provide merge-time and recurring feedback through GitHub Actions, and support the wider application security assessment together with SAST and manual pentesting.

Scope

The dynamic analysis scope covers the running ClinicWave application deployed in an ephemeral Docker-based environment for testing purposes. This includes the React and Vite frontend, the Spring Boot backend, the PostgreSQL database required for application startup, the HTTP routes exposed by the frontend, and the backend endpoints reachable through the application during scanning. Rather than inspecting source files directly, DAST focuses on the behavior of the live application once it is built, started, and exposed to the scanner.

Two-Level DAST Strategy

Pull Request Level

The first DAST level runs on pull requests through dast-pr.yml. This is the merge-time runtime security gate. The workflow builds the frontend and backend from the pull request branch, starts a temporary PostgreSQL container, brings the full application up inside a dedicated Docker network, waits for both the frontend and backend to become reachable, and then runs an OWASP ZAP Baseline scan against the frontend entry point.

At this level, the objective is to detect obvious runtime security issues before code is merged. The workflow generates both HTML and JSON ZAP reports, uploads them as workflow artifacts, evaluates the JSON report through a script, and publishes a DAST summary comment back to the pull request. The PR gate is configured to fail when the ZAP report contains High severity findings, which makes this level a practical merge-time control rather than just an informational scan.

Nightly Level

The second DAST level runs through dast-nightly.yml. This workflow is defined in the AppSec repository and checks out the software repository separately, by default scanning the develop branch unless another target reference is provided manually. As in the PR workflow, the application is built and started inside an isolated Docker environment with PostgreSQL, backend, and frontend services, and then scanned with OWASP ZAP Baseline.

The purpose of the nightly level is to provide recurring runtime coverage outside the time pressure of a single pull request. It produces HTML and JSON ZAP reports, uploads them as artifacts, and writes a summarized view of the alert counts and alert types into the workflow summary. This level is useful for continuous visibility, because it can detect issues that remain present over time even if they were not the focus of a specific pull request review.

Why the Two Levels Matter

The two DAST levels serve different but complementary purposes. The pull request level provides merge-time validation and direct reviewer visibility, while the nightly level provides recurring security monitoring against a stable target branch. This separation is useful because runtime scanning is slower and more environment-dependent than static analysis, so it benefits from having one layer optimized for gating and another optimized for ongoing observation.

Runtime Environment and Tooling

The DAST process in this project is centered on OWASP ZAP Baseline. In both workflows, the application is launched inside Docker with a temporary PostgreSQL container, a backend container exposing the Spring Boot API, and a frontend container exposing the React and Vite application. The scanner targets the running frontend and follows the application from there, which allows it to observe the real runtime surface instead of only the source configuration.

Strengths and Limitations

One of the main strengths of this model is that it validates the application in a running state rather than only inspecting code or configuration. This makes it useful for identifying issues that are visible only after deployment, such as missing headers, weak runtime defaults, or exposed routes and pages. The use of GitHub Actions artifacts, workflow summaries, and PR comments also makes the results easier to review and include as evidence in the report.

At the same time, DAST has clear limitations. It depends on the application starting correctly in the test environment, and the quality of the results depends on how much of the application surface the scanner can actually reach. A baseline scan is also more effective at identifying broad web security issues than deep business-logic flaws. For that reason, DAST should be interpreted as one part of a broader security process rather than as a complete assessment on its own.

Results

This section should be completed by presenting evidence from the two DAST levels.

Pull Request Results

During pull request validation, the repository executed the DAST workflow to assess the running application before merge. The workflow built the backend and frontend containers, started the supporting PostgreSQL database, waited for the application to become reachable, and then ran an OWASP ZAP Baseline scan against the frontend. The workflow generated both HTML and JSON reports, uploaded them as artifacts, evaluated the JSON output against the configured gate, and posted a DAST summary comment back to the pull request. In this setup, the most important point to record is whether the scan completed successfully, whether any High severity alerts were found, and whether the pull request gate passed or failed as a result.

In the latest pull request run, the DAST workflow completed successfully and the gate passed because no **High** severity ZAP findings were detected. The scan reported 2 medium, 7 low, and 12 informational alerts, mainly related to missing security headers such as CSP and anti-clickjacking protections. The workflow also generated report artifacts and posted a DAST summary comment back to the pull request, which gave reviewers a clear view of the runtime security results directly in GitHub.

DAST Scan Summary

- Target ref: `develop`

Severity	Count
High	0
Medium	2
Low	7
Informational	12
Total	21

Alert Types

Alert	Severity	Instances
Content Security Policy (CSP) Header Not Set	Medium	4
Missing Anti-clickjacking Header	Medium	4
Permissions Policy Header Not Set	Low	5
X-Content-Type-Options Header Missing	Low	5
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	4
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	4
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	4
Timestamp Disclosure - Unix	Low	2
Strict-Transport-Security Header Not Set	Low	1
Base64 Disclosure	Informational	10
Information Disclosure - Suspicious Comments	Informational	5
Storable but Non-Cacheable Content	Informational	5
Modern Web Application	Informational	4
Sec-Fetch-Dest Header is Missing	Informational	4
Sec-Fetch-Mode Header is Missing	Informational	4
Sec-Fetch-Site Header is Missing	Informational	4
Sec-Fetch-User Header is Missing	Informational	4
Information Disclosure - Sensitive Information in URL	Informational	1
Re-examine Cache-control Directives	Informational	1
Retrieved from Cache	Informational	1
Storable and Cacheable Content	Informational	1

Workflow Links

- Open workflow run and artifacts

Gate Status: Passed. No blocking ZAP findings detected.

PR Artifacts

ZAP Scanning Report

Sites: https://firefox-settings-attachments.cdn.mozilla.net http://frontend:5173

Generated on Mon, 30 Mar 2026 21:50:45

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	7
Informational	12
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	13
Info	Informational	http://frontend:5173	Percentage of responses with status code 1xx	2 %
Info	Informational	http://frontend:5173	Percentage of responses with status code 2xx	40 %
Info	Informational	http://frontend:5173	Percentage of responses with status code 3xx	56 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type application/json	12 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type image/svg+xml	2 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type text/html	6 %
Info	Informational	http://frontend:5173	Percentage of endpoints with content type text/javascript	70 %
Info	Informational	http://frontend:5173	Percentage of endpoints with method DELETE	2 %
Info	Informational	http://frontend:5173	Percentage of endpoints with method GET	91 %
Info	Informational	http://frontend:5173	Percentage of endpoints with method PUT	6 %
Info	Informational	http://frontend:5173	Count of total endpoints	48
Info	Informational	http://frontend:5173	Percentage of slow responses	3 %
Info	Informational	https://firefox-settings-attachments.cdn.mozilla.net	Percentage of responses with status code 2xx	100 %
Info	Informational	https://firefox-settings-attachments.cdn.mozilla.net	Percentage of endpoints with content type text/plain	100 %
Info	Informational	https://firefox-settings-attachments.cdn.mozilla.net	Percentage of endpoints with method GET	100 %

Alert Detail

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://frontend:5173
Node Name	http://frontend:5173
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://frontend:5173/
Node Name	http://frontend:5173/
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://frontend:5173/robots.txt
Node Name	http://frontend:5173/robots.txt
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://frontend:5173/sitemap.xml
Node Name	http://frontend:5173/sitemap.xml
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
Instances	4

Nightly Results

The nightly DAST workflow provided the deeper recurring runtime analysis of the application. It checked out the software repository, built and launched the full stack inside Docker, ran OWASP ZAP Baseline against the running frontend, and then uploaded the resulting reports as artifacts. In addition to the raw reports, the workflow produced a summary of alert counts by severity and a breakdown of alert types in a summary. This level is useful because it provides a recurring view of the runtime security posture of the project even when no pull request is being reviewed.

Nightly ZAP Summary and Artifacts

DAST Scan Summary

- Target repository: `detiuaveiro/softwate-baseline-ses25_102`
- Target ref: `develop`

Severity	Count
High	0
Medium	2
Low	8
Informational	12
Total	22

Alert Types

Alert	Severity	Instances
Content Security Policy (CSP) Header Not Set	Medium	4
Missing Anti-clickjacking Header	Medium	4
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	5
Permissions Policy Header Not Set	Low	5
X-Content-Type-Options Header Missing	Low	5
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	4
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	4
Application Error Disclosure	Low	1
Strict-Transport-Security Header Not Set	Low	1
Timestamp Disclosure - Unix	Low	1
Base64 Disclosure	Informational	11
Information Disclosure - Suspicious Comments	Informational	5
Storable but Non-Cacheable Content	Informational	5
Modern Web Application	Informational	4
Sec-Fetch-Dest Header is Missing	Informational	3
Sec-Fetch-Mode Header is Missing	Informational	3
Sec-Fetch-Site Header is Missing	Informational	3
Sec-Fetch-User Header is Missing	Informational	3
Information Disclosure - Sensitive Information in URL	Informational	1
Re-examine Cache-control Directives	Informational	1
Retrieved from Cache	Informational	1
Storable and Cacheable Content	Informational	1

ZAP by Checkmarx

Sites: https://firefox-settings-attachments.cdn.mozilla.net http://frontend:5173

Generated on Sun, 29 Mar 2026 14:56:00

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	8
Informational	12
False Positives:	0